

Com S 3270

Advanced Programming Techniques

Spring 2026

Updated: 06:37 Thursday 22nd January, 2026

Instructors

Jeremy Sheaffer

Office 113 Durham
Email sheaffer@iastate.edu (please preface all email subjects with “CS 3270: ”)
Section 1 TR 9:30–10:45 in Library 0198
Office Hours M 10:15–11:45
T 7:45–9:15

Trent Muhr

Office 130 Atanasoff
Email muhr@iastate.edu (please preface all email subjects with “CS 3270: ”)
Section 2 TR 12:40–13:55 in Marston 2300
Section 3 TR 11:00–12:15 in Marston 2300
Office Hours M 12:00-14:00

Teaching Assistants

Name	Email	Office Hours	Location
Daniel Asante	dasante@iastate.edu	W 12:00-16:00	Pearson 0145
Koushik Howlader	howlader@iastate.edu	R 16:00-18:00	Pearson 0145
Tasiful Islam	islam093@iastate.edu	F 12:00-14:00	Pearson 0145
Asif Siddique	asif64@iastate.edu	M F 14:00-16:00	Pearson 0145
Samrajya (Sam) Thapa	svthapa@iastate.edu	T R 14:00-16:00	Pearson 0145

Course Summary (from the catalog)

Object-oriented programming experience using a language suitable for exploring advanced topics in programming. Topics include memory management, parameter passing, inheritance, compiling, debugging, and maintaining programs. Significant programming projects.

Course Summary (from the instructor)

We will cover the topics described above with two languages, C and C++, used as drivers. We will spend roughly half of the course on C and half on C++. We will also devote some time to programming environment, build tools, version control, and debugging; these are all concepts that you will need to understand and employ to be successful in this course and beyond.

Course Objectives

After successfully completing this course, students will:

- understand differences between managed languages (e.g., Java) and unmanaged languages (e.g., C and C++);
- understand and be able to use simple build systems (e.g., make);
- be able to design and build large programs from specification;
- be able to use third-party libraries in programs;
- be able to read, write, and modify C programs;
- understand memory management techniques for C;
- be able to read, write, and modify C++ programs;
- understand memory management techniques for C++;
- understand C++ templates and the standard template library.

Course Outcomes

This course has three major ABET outcomes:

1. By the end of this course, students will be able to produce efficient and correct C and C++ programs of significant lengths from specifications.
2. By the end of this course, students will be able to understand and use advanced C and C++ features in software development and maintenance.
3. By the end of this course, students will be able to write and debug large C and C++ programs based on English descriptions or pseudocode.

Prerequisites

Course prerequisites are Com S 2280 and credit and Math 1650, or instructor permission. No exceptions. If you do not meet these prerequisites, it is the policy of the Computer Science department that you will be automatically unenrolled from the course.

Textbooks

There are no required texts for this course. All required material will be presented in lecture. Supplemental material may be found in the following excellent texts:

- Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*.
- Bjarne Stroustrup, *The C++ Programming Language*.
- Erich Gamma, John Vlissides, Ralph Johnson, and Richard Helm, *Design Patterns: Elements of Reusable Object-Oriented Software*.

Recommended Software

For evaluation purposes, we require a uniform platform where the TAs can build and run your software. This platform is `pyrite.cs.iastate.edu`. If your submissions do not work on Pyrite, you will not receive full credit for your work.

In recent years, students have mainly used one of two methods for development in this class:

1. VSCode as a frontend to Pyrite via ssh (requires a network connection).
2. Using the *Windows Subsystem for Linux* (WSL) to work locally on a Windows host.

Students who use Macs can install gcc and work locally. Students who run Linux natively or in a VM are unlikely to require guidance in setting up a development environment.

The VSCode options requires that you mount your *U drive*. Even if you are not using VSCode, mounting your U drive is a convenient, useful thing to do. See <https://etg.ece.iastate.edu/file-storage/>.

Remote Server

It is always possible to complete the programming assignments by working directly on `pyrite.cs.iastate.edu` or on any of the Linux machines in the labs. Connect to pyrite using ssh. A nice, free Windows ssh client is PuTTY (<http://www.putty.org/>). To resolve pyrite from off campus, you will need to first connect to the Iowa State VPN (<https://vpn.iastate.edu/>).

Other Necessities

In a Linux environment, you can edit with Emacs, vi, or Pico (the first two are powerful and used by professional programmers all over the world, while the latter is simple and used by undergraduate computer science students with tunnel vision all over the world).

Regardless of your working environment, you will need a C and a C++ compiler, an editor, and a debugger. You may use whatever specific tools you like, so long as your submitted code compiles and runs on one of the VMs. In lecture, I will use GCC and G++ for compiling, Emacs for editing, and GDB for debugging, because these are what I use for real programming.

Be careful if you use Visual Studio! Visual Studio will often compile code that does not compile under GCC. If you are not developing on pyrite or one of the VMs, be sure to give yourself enough time for testing before you must submit.

The main project will use the ncurses library, which may be obtained in many different ways. Official site: <http://www.gnu.org/software/ncurses/ncurses.html>. The Slackware VM has the necessary ncurses development libraries; on the Fedora VM, you will need to issue the following command as root: `dnf install ncurses-devel`.

All submissions must compile with GNU Make for credit. Official site: <http://www.gnu.org/software/make/>

Helpful Extras

Version control software, such as `git` or `subversion`. Among other things, this allows you to keep your source code synchronized between your personal machine and the testing machine.

- Subversion book: <http://svnbook.red-bean.com/>
- Git book: <http://git-scm.com/book>

Valgrind (<http://valgrind.org/>) or some other memory profiling tool. This can save *hours* of work tracking down segmentation faults.

Lecture Attendance

Attendance is required and we will take roll beginning second week of class. You are responsible for all material and announcements in lecture. Often, things are announced in lecture well before they are posted to Piazza, if they are posted at all.

[illegible]

Screenshot from a game of NetHack, showing a message on top (the player has attacked a mind flayer, a very dangerous monster that will suck out your brains), dungeon view in the middle, including the character (the '@'), the mind flayer (the 'h'), gold ('\$'), various monsters that the character is aware of magically (the numbers, which denote their approximate strengths), a moat (the many '}''), many weapons, pieces of armor, gems, traps, doors, corridors, and items of food, and, on the bottom, the character's status.

Canvas and Piazza

We will be using Canvas and Piazza for this course. In particular,

- Canvas will be used for assignment submissions and the gradebook; and
- all other online course content will be managed through Piazza.

We expect you to check both Canvas and Piazza regularly for announcements and homework or exam information. It is your responsibility to know how to log in, read announcements, use the discussion boards, and submit homework.

Course Work

Projects (24%)

Over the course of the semester, we will be building a Pokémon-based Roguelike game using ASCII graphics. Two iconic modern examples of Roguelikes are Angband and NetHack. The figure shows a screen shot of a typical NetHack game. Your games will probably not be nearly as extensive as these two examples—unless you want them to be!—but by the end of the semester, you will have developed a playable game with, at minimum, nice-looking, randomly-generated maps, trainer and Pokémon battles, wild Pokémon spawns, capture, and level-up. Your game will be far from implementing all of the mechanics in Pokémon games, as this would be lengthy, tedious, and non-instructive, but it will be playable and—if not fun—amusing.

Certain functionality will be due, beginning the second week of classes, every week except for Fall Break and Dead Week at 10:00pm on Wednesdays. Project requirements will be specified in advance, so that you may work ahead, but at any given time, you will only be graded on the functionality required by that date.

All assignments must be turned in with an ASCII readme file, named README and containing no more than 150 words, which describes your project's compliance with the specification to date including relevant function, data structures, and file names. Assignments must also contain an ASCII change log file named CHANGELOG that contains a cumulative list of dated updates from day 1 (e.g., "started project"). The change log will list every significant change you make to your project, including bug discoveries, bug fixes, functionality additions and changes, etc.

All projects must include a working makefile, named Makefile, which correctly builds a binary with debugging symbols on pyrite. We will discuss these things in the first week of class.

All projects must be submitted via the appropriate assignment page on Canvas. Projects should be turned in as a *tarball* (a gzipped tar archive). Basic instructions, with an example, on how to create and extract acceptable tarballs follow. Tarballs are to be named

```
<surname>_<forename>.assignment-<assignment number>.tar.gz
```

and should extract to a directory named

```
<surname>_<forename>.assignment-<assignment number>
```

where <surname> is your family name, <forename> is your given name, <assignment number> is the submitted assignment number, and all other characters are literals. Thus, if I am submitting assignment 1.05, my tarball will be named

```
sheaffer_jeremy.assignment-1.05.tar.gz
```

and it will expand to the directory

```
sheaffer_jeremy.assignment-1.05
```

which will contain all of my source files, and Makefile, README, and CHANGELOG as defined above, but no generated files (run `make clean` first!).

Still using assignment 1.05 as the driving example, the simplest way to create a tarball follows. This is an actual session from my shell, saved with a program named `script`. I have added comments inline, preceding them with ‘%%

, which are not part of the actual shell I/O. Lines beginning with sheaffer@sheaffer... are input. Lines ending with a backslash ('\') are continued on the following line. I've indented continued lines by two spaces to make them stand out. All other lines are output.

```
Script started on Thu 08 Jan 2015 08:20:53 AM CST
%% Run 'make clean' to get rid of the binaries
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments/assignment1.05$ make clean
rm -f dungeon.o cds/heap.o rlg229
%% Move into the parent directory so that we can make a tarball of the
%% current directory
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments/assignment1.05$ cd ..
%% My directory isn't named as per the specification, so create a copy of it.
%% '-R' makes it copy recursively (a deep copy) so the 'cds' subdirectory is
%% copied, too, or you could rename it with 'mv'.
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ cp -R assignment1.05/ \
    sheaffer_jeremy.assignment-1.05
%% Create the tarball. Four switches to tar. 'c' means create, 'v' means
%% verbose, 'f' means force, and 'z' means compress with gzip. 'v' and 'f'
%% aren't strictly needed, but 'v' does cause it to give the file list that
%% follows.
%% Note that the backslash ending the next line is to escape the newline in
%% a multi-line command. It is not part of the command!
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ tar cvfz \
    sheaffer_jeremy.assignment-1.05.tar.gz sheaffer_jeremy.assignment-1.05/
sheaffer_jeremy.assignment-1.05/
sheaffer_jeremy.assignment-1.05/cds/
sheaffer_jeremy.assignment-1.05/cds/hash.h
sheaffer_jeremy.assignment-1.05/cds/tree.c
sheaffer_jeremy.assignment-1.05/cds/macros.h
sheaffer_jeremy.assignment-1.05/cds/neural_net.h
sheaffer_jeremy.assignment-1.05/cds/neural_net.c
sheaffer_jeremy.assignment-1.05/cds/list.c
sheaffer_jeremy.assignment-1.05/cds/heap.c
sheaffer_jeremy.assignment-1.05/cds/list.h
sheaffer_jeremy.assignment-1.05/cds/spinlock.h
sheaffer_jeremy.assignment-1.05/cds/heap.h
sheaffer_jeremy.assignment-1.05/cds/hash.c
sheaffer_jeremy.assignment-1.05/cds/heap.d
sheaffer_jeremy.assignment-1.05/cds/circular_queue.h
sheaffer_jeremy.assignment-1.05/cds/circular_queue.c
sheaffer_jeremy.assignment-1.05/cds/tree.h
sheaffer_jeremy.assignment-1.05/Makefile
sheaffer_jeremy.assignment-1.05/dungeon.h
sheaffer_jeremy.assignment-1.05/dungeon.c
sheaffer_jeremy.assignment-1.05/CHANGELOG
sheaffer_jeremy.assignment-1.05/README
%% At this point, I am done. I could submit, however, I don't want to risk a
%% penalty to my grade, so I am going to test it out. I can't really expand
%% the archive here, because the target directory exists. Since that
%% directory is a copy, I could just delete it now, but if I made a mistake
%% and need to try again, I would rather not have to retype all that stuff, so
%% let's make a temporary directory to expand in, call it 'test'.
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ mkdir test
%% And move into it...
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ cd test
%% Now to open up the archive. gzip compresses and decompresses. The 'd'
```

```

%% switch tells it to decompress. 'c' tells it to concatenate the output to
%% the standard output, then we pipe ('|') that to tar, where 'x' means to
%% expand ('v' and 'f' are as above) and a file name of '-' means to read the
%% file from the standard input. I can see in the output that follows that
%% the files are all there. I can move in, do a build, some testing, etc.,
%% as I please.
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments/test$ gzip -dc \
  ../sheaffer_jeremy.assignment-1.05.tar.gz |tar xvf -
sheaffer_jeremy.assignment-1.05/
sheaffer_jeremy.assignment-1.05/cds/
sheaffer_jeremy.assignment-1.05/cds/hash.h
sheaffer_jeremy.assignment-1.05/cds/tree.c
sheaffer_jeremy.assignment-1.05/cds/macros.h
sheaffer_jeremy.assignment-1.05/cds/neural_net.h
sheaffer_jeremy.assignment-1.05/cds/neural_net.c
sheaffer_jeremy.assignment-1.05/cds/list.c
sheaffer_jeremy.assignment-1.05/cds/heap.c
sheaffer_jeremy.assignment-1.05/cds/list.h
sheaffer_jeremy.assignment-1.05/cds/spinlock.h
sheaffer_jeremy.assignment-1.05/cds/heap.h
sheaffer_jeremy.assignment-1.05/cds/hash.c
sheaffer_jeremy.assignment-1.05/cds/heap.d
sheaffer_jeremy.assignment-1.05/cds/circular_queue.h
sheaffer_jeremy.assignment-1.05/cds/circular_queue.c
sheaffer_jeremy.assignment-1.05/cds/tree.h
sheaffer_jeremy.assignment-1.05/Makefile
sheaffer_jeremy.assignment-1.05/dungeon.h
sheaffer_jeremy.assignment-1.05/dungeon.c
sheaffer_jeremy.assignment-1.05/CHANGELOG
sheaffer_jeremy.assignment-1.05/README
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments/test$ cd ..
%% Looks good! I am ready to submit. I don't need the temporary directory,
%% nor the renamed directory, so I am cleaning them up.
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ rm -rf test
sheaffer_jeremy.assignment-1.05
%% And exiting the 'script' program.
sheaffer@sheaffer-e7440:~/classes/semester/s2015/229/assignments$ exit

```

Script done on Thu 08 Jan 2015 08:22:54 AM CST

Given the cumulative nature of the assignments, we will attempt to return all assignments, except assignment 12, within 48 hours. **Assignments submitted with formats that do not match the above specification will receive a penalty of at least 20%. A assignment without an up-to-date readme, an up-to-date changelog, and a working Makefile will not be graded and will receive a grade of 0. A required feature which is not discussed in the readme will not be evaluated, and thus will not receive credit.** Since the assignment is cumulative, it will grow to such a size that searching your code for required features will become intractable to the graders. These rules are necessary to make grading possible.

Because the assignments are cumulative and we do not want students to fall behind with no reasonable chance of catching up, the instructor will periodically release code checkpoints to the class which may be used in part or in whole by any student in the class in subsequent submissions.

The first assignment is special, intended to get you acquainted with the development environment and to provide a gentle introduction to C, and it is not part of your game. For this reason, we will start numbering with 0.

All assignments will be graded out of 10 points. Some assignments may be extended, reducing the total number of assignments; thus, the final weight of each assignment will be 24% divided by the final number of assignments.

Assignment	Due Date
Assignment 0	Wed Jan 28 22:00:00 CST 2018
Assignment 1.01	Wed Feb 4 22:00:00 CST 2018
Assignment 1.02	Wed Feb 11 22:00:00 CST 2018
Assignment 1.03	Wed Feb 18 22:00:00 CST 2018
Assignment 1.04	Wed Feb 25 22:00:00 CST 2018
Assignment 1.05	Wed Mar 4 22:00:00 CST 2018
Assignment 1.06	Wed Mar 11 22:00:00 CDT 2018
Assignment 1.07	Wed Mar 25 22:00:00 CDT 2018
Assignment 1.08	Wed Apr 1 22:00:00 CDT 2018
Assignment 1.09	Wed Apr 8 22:00:00 CDT 2018
Assignment 1.10	Wed Apr 15 22:00:00 CDT 2018
Assignment 1.11	Wed Apr 23 22:00:00 CDT 2018
Assignment 2	Wed Apr 29 22:00:00 CDT 2018

Exams (66%)

There will be two exams, a midterm and a final. Each exam is 33% of your total grade. Exams will be held in the testing center. Details to follow.

Attendance (10%)

Roll will be taken each class meeting beginning in week two. You will receive up to four absences without penalty (two full weeks of class). These free absences apply whether or not you have a “good” excuse, so please do not contact the instructors or TAs about your absence. Your attendance grade will be your percentage of attended lectures, less the free misses.

Lateness Policy

Work is due on the specified due date. No late work will be accepted.

Grading

Midterm and final grades will be curved. The curves will not be determined until grades are calculated, and will be designed to fairly rank your performance against that of your peers and reward you appropriately for your work. A traditional 10-point scale establishes a lower bound on your grade now. Any curve that is applied at the midterm will establish a new lower bound for the final grade, which may in turn be curved even more in your favor.

Bug Bounty

All solution code that is released by the instructor to Canvas is subject to a bug bounty. The bug bounty works as follows:

Solutions will contain a BUGS file listing known bugs. These bugs are not eligible for bug bounties.

When you discover what you believe to be a bug in solution code, you must:

- find an input that reproduces that bug;
- write a fix for the bug which is itself (apparently) bug free;

- create a diff for your bug fix against the bugged solution release, suitable for use with the patch command; and
- post all of this as a comment to the respective solution thread on Canvas.

If you are the first to post a correct bug fix, you will be rewarded with an increase of one grade level (that is, A- becomes A, C+ becomes B-, etc., but no change for an A) to your final course grade.

Just because something doesn't work the way you think it should doesn't make it a bug. I reserve the right to reject your submission if it "repairs" something that I consider not a bug. You may ask me before you work if I would consider *X* to be a bug; I will give you the best answer I can given the accuracy and detail of your question.

Be advised that if you post an incorrect bug report, input, or fix, the information immediately becomes available to your classmates, who may then beat you with the correction, so try to get it right the first time.

Bug bounties will not be evaluated until the end of the semester.

Academic Integrity and Generative AI

Exams and any and all required software functionality are to be your own work. We will take academic dishonesty seriously. We will report it to the Dean of Students. If you are found responsible for academic dishonesty in this class, you will receive a failing mark for the course. We believe that you are mature enough to understand "integrity" without a great deal of detail in the syllabus. Here we list some examples of things which you may and may not do in this class. If you are ever unsure, discuss it with the instructor.

The following examples should be taken in spirit, not as a complete list. For example, you may not edit another student's code, so it should be clear that you also may not write code on paper for another student to use. With respect to programming assignments, you may not:

- Share required elements of your project source code with anyone before the instructor clears those elements for sharing.
- Study required elements of another student's source code.
- Submit another student's work as your own.
- Debug another student's source code.
- Edit another student's code.
- Use code from the Web or elsewhere to satisfy required functionality in your project before submission for that functionality is closed.
- Use third party code in your project without attribution.

You may:

- Discuss algorithms, data structures, and design with other students and on Canvas
- Directly use third-party code for non-required functionality with complete and correct attribution
- Directly use third-party code for required functionality, provided submission for that functionality has already been closed and you completely and correctly attribute the author.
- Implement algorithms learned from third-party sources in your code
- Share code that implements non-required functionality on Canvas.
- Share code that implements required functionality on Canvas after the instructor gives clearance to do so.
- Help another student debug, so long as that student "drives."

The University's policies on academic dishonesty are located online at:

<http://catalog.iastate.edu/academiclife/regulations/#academicdishonestytext>.

Please speak with the instructor directly if you have any questions or doubts.

All code submitted for a grade is to be your own work, which means it may not be generated by AI. You're welcome to use AI as a study aid, friend, or life partner, per your choosing, but you may not submit AI output as your own work.

Disabilities

Iowa State University complies with the Americans with Disabilities Act and Section 504 of the Rehabilitation Act. Any student who may require an accommodation under such provisions should contact the instructor as soon as possible and no later than the end of the first week of class or as soon as you become aware. Please obtain a SAAR form verifying your disability and specifying the accommodation you will need. No retrospective accommodations will be required in this class.

Harassment and Discrimination

Iowa State University strives to maintain our campus as a place of work and study for faculty, staff, and students that is free of all forms of prohibited discrimination and harassment based upon race, ethnicity, sex (including sexual assault), pregnancy, color, religion, national origin, physical or mental disability, age, marital status, sexual orientation, gender identity, genetic information, or status as a U.S. veteran. Any student who has concerns about such behavior should contact his or her instructor, Student Assistance at 515-294-1020 or email dso-sas@iastate.edu, or the Office of Equal Opportunity and Compliance at 515-294-7612.

Religious Accommodation

If an academic or work requirement conflicts with your religious practices or observances, you may request reasonable accommodations. Your request must be in writing, and your instructor or supervisor will review the request. You or your instructor may also seek assistance from the Dean of Students Office or the Office of Equal Opportunity and Compliance.